

Will Earlier Projects Plus a Disciplined Process Enforce SE Principles Throughout the CS Curriculum?

Linda B. Sherrell
The University of Memphis
373 Dunn Hall
Memphis, TN 38152, USA
+1-901-678-3139
sherrell@cs.memphis.edu

Sajjan G. Shiva
The University of Memphis
209 Dunn Hall
Memphis, TN 38152, USA
+1-901-678-5465
sshiva@memphis.edu

ABSTRACT

This paper discusses two related challenges faced by software engineering instructors. First, assuming that projects are necessary to produce successful computer science majors, what should be the role of projects and how best do we integrate theory and application? Second, what life cycle models and associated processes should students have the opportunity to experience, and where in the curriculum should a disciplined process first appear? We review several curriculum plans that have been employed to address these problems. We also offer recommendations based on our experiences both with undergraduate computer science majors and with high school students in project *Tri-P-LETS*, where beginning programming students are taught to develop games and software simulations following a process.

Categories and Subject Descriptors

K.3.2 [Computers and Education]: Computer and Information Science Education – computer science education, curriculum, GK-12; K.3.6 [Software Engineering, Management of Computer and Information Systems]: Software Management – software development, software process.

General Terms

Management, Measurement, Documentation.

Keywords

Unified Process, agile methods, *Tri-P-LETS*, AgentSheets, PSP, high school programming courses.

1. THE NEED FOR PROJECTS

Team projects are a necessary component of software engineering courses to prepare computer science students for careers as software engineers. Industry recruiters for software development positions routinely ask applicants about their exposure to team

environments. Prior experience influences the success of software projects, where background knowledge may result from the development of a similar software system or the application of a standard process. But how are students to be successful developing their first team project when they have no prior experience and are unfamiliar with software engineering processes?

One approach that several universities have adopted is to teach a two-course sequence in software engineering, where theory is taught in the first course and students develop a capstone project in the second course [6]. Certainly, this approach allows for a more meaningful project, but should this be the formula for all instructors to follow? Based on our experiences, students enjoy and learn more in the first course when there is a project. For example, students can prepare parts of analysis and design documents using modeling techniques taught in the course; they can possibly implement a system prototype. Then, in the capstone course, students incrementally go through the requirements, analysis, design, implementation, and testing workflows to produce a complete product.

However, one can argue that when students are ineligible for the two-course sequence until their senior year, they do not receive enough experience using a disciplined process, and will not have the opportunity to use the project/process concepts in other courses that they take. Furthermore, students are often entrenched in certain development modes, and it may be difficult to break poor habits. To help alleviate this problem, Computing Curricula 2001 [2] encourages instructors to assign a medium-sized project in the second or third introductory programming course. Given the project requirements, students can experience the other major phases of a software life cycle and develop team skills. Note that Computing Curricula guidelines do not mention metrics for this course. A more empirical approach would require students to use the Personal Software Process (PSP) [3], where the associated forms can be introduced in CS1.

Other possible questions for participants in this session include the following. What types of projects should be assigned at the freshman and sophomore level? Should students be required to implement web-based systems; if so, when in the curriculum? What steps can be taken so that students will apply software engineering principles in all courses requiring projects, not just those taught by software engineering specialists or enthusiasts?

2. THE NEED FOR EARLY PROCESS

In beginning programming courses, students are taught to describe an algorithm using natural language or pseudocode and then to apply stepwise refinement when developing a program. However, many students resist all notions of using even such a simple process, anxious to get to code as quickly as possible. We have experienced this phenomenon with both novice programmers and graduate students. Students have been known to successfully build a system using a software engineering process in a capstone course and then later revert to bad habits in graduate courses. We have heard comments from students such as “the techniques that are taught in the software engineering course are not applicable in the real world.” Furthermore, non-software engineering CS faculty may be unfamiliar with modern software engineering practices, and unknowingly propagate an undisciplined approach to software development.

What can be done to rectify this situation? Students can be exposed to multiple software development life cycle models and processes throughout the curriculum. Although most software engineering textbooks discuss a variety of life cycle models, unless students have the opportunity to experience some of the more popular processes firsthand, they will not gain a true perspective of the strengths and weaknesses of approaches.

At the University of Memphis, we have modified our curriculum to provide students with hands-on experience using several different process models. In CS3:Advanced Data Structures and Algorithms, we use a modified waterfall model with feedback loops to introduce the major phases of software development. However, students are encouraged to code and test their modules using an incremental, iterative approach. Next, in the first software engineering course, students are taught the fundamentals of software engineering [6] along with the Unified Process (UP). Students gain an appreciation for business phases and technical workflows by specifying, designing, and writing test plans for a significant project. The project emphasizes project management, modeling with UML, and the use of CASE tools as opposed to implementation. Finally students develop and implement a capstone project in their final semester using the Unified Process. At the present time, undergraduate students also have the opportunity to work in multidisciplinary teams using agile software development methods in our Tools for Bioinformatics course. We also offer the graduate course Software Development Process Models, where students are exposed to a variety of agile and planned methods [1]. In the graduate course, students develop a project using an agile process and compare their experiences with the Unified Process and other processes.

But what about students who now take programming courses in high school? These students are often the most resistant to the use of good software engineering practices since they have been successful in their past programming courses without using a disciplined process. Usually this is because the assigned programs were small enough that the students found it unnecessary to use a process. Furthermore, many high school teachers do not have degrees in computer science and have not completed courses in software engineering so they are unfamiliar with software life cycle models and associated processes. Often too much time is spent on teaching syntax as opposed to algorithmic development and general programming concepts.

3. THE TRI-P-LETS PROJECT

At the University of Memphis, we are working on *Tri-P-LETS* (Three P Learning Environment for Teachers and Students) [7]. The aim of this project is to enhance the curriculum in local high school programming courses thereby providing students with a better understanding of the field of computer science including how good software should be developed. Fellows majoring in computer science assist participating teachers by presenting modules that emphasize important concepts such as problem-solving, object-oriented design, teamwork, and software development processes.

Students learn to build software simulations in AgentSheets [4] using a process to identify agents and their behaviors, providing a foundation for later work in object-oriented programming languages. Students also record the times spent on different tasks and note their programming errors on forms similar to those of PSP, but simplified to better fit the knowledge level of the high school students. Because students are able to build meaningful and fun projects in AgentSheets [5], their enthusiasm for programming is reinforced and the advantages of using a disciplined process are demonstrated. The three P's: problem-solving, programming concepts, and process application continue to be emphasized by the Fellow/teacher pairs as students move from programming in AgentSheets to Visual Basic.Net to Java.

Throughout the *Tri-P-LETS* project, students are given the opportunity to experience the benefits of pair programming and to work in larger team environments. Near the end of each academic year, students participate in a two-month Programming Challenge in which they work in teams to design, implement, test, and document a significant software project. Projects are judged by representatives from academia and industry. Since students in project *Tri-P-LETS* will use a process throughout the high school programming curriculum (up to three courses), following a process will hopefully become routine.

4. ACKNOWLEDGEMENTS

Project *Tri-P-LETS* is supported by National Science Foundation GK-12 Grant “Computer Science Resources for Memphis Area High Schools” (DGE-0338334).

5. REFERENCES

- [1] Boehm, B. and Turner, R. *Balancing Agility and Discipline: A Guide for the Perplexed*, Addison Wesley, Boston, 2004.
- [2] *Computing Curricular 2001*, available at: <http://www.computer.org/education/cc2001/cc2001.pdf>
- [3] Humphrey, W.S. *Introduction to the Personal Software Process*, Addison Wesley, Boston, MA, 1997.
- [4] Repenning, A. AgentSheets, available at: <http://www.agentsheets.com>.
- [5] Repenning, A. (2004). Agent-based End-user Development. *Communications of the ACM*. 47(9), 43-46.
- [6] Schach, S. R. *Object-Oriented and Classical Software Engineering*. McGraw Hill, Boston, MA, 2005.
- [7] *Tri-P-LETS* website. See: <http://classweb.cs.memphis.edu/~hsfellow>.